

## 1. Combining Customer Transaction and Demographic DataFrames

### Objective

We need to combine two large DataFrames while ensuring:

- No duplicate Customer\_ID
- Consistent indexing after merging
- All valid records are preserved
- Missing or mismatched records are identified
- Data is validated using filtering, grouping, sorting, and indexing

### Step 1: Import Libraries and Create Sample Data

```
import pandas as pd
import numpy as np
# Customer transaction data
transactions = pd.DataFrame({
    'Customer_ID': [101, 102, 103, 104, 105, 105],
    'Transaction_Amount': [500, 1200, 750, 300, 900, 900],
    'Transaction_Date': [
        '2026-08-01',
        '2026-08-02',
        '2026-08-03',
        '2026-08-04',
        '2026-08-05',
        '2026-08-05'
    ]
})

# Customer demographic data
demographics = pd.DataFrame({
    'Customer_ID': [101, 102, 103, 104, 106],
    'Name': ['Arun', 'Bala', 'Charan', 'Divya', 'Eshan'],
    'Age': [25, 30, 28, 35, 40],
    'City': ['Chennai', 'Mumbai', 'Delhi', 'Chennai', 'Bangalore']
})

print("Transaction Data:")
display(transactions)

print("Demographic Data:")
display(demographics)
```

Transaction Data:

|   | Customer_ID | Transaction_Amount | Transaction_Date |
|---|-------------|--------------------|------------------|
| 0 | 101         | 500                | 2026-08-01       |
| 1 | 102         | 1200               | 2026-08-02       |
| 2 | 103         | 750                | 2026-08-03       |
| 3 | 104         | 300                | 2026-08-04       |
| 4 | 105         | 900                | 2026-08-05       |
| 5 | 105         | 900                | 2026-08-05       |

Demographic Data:

|   | Customer_ID | Name   | Age | City      |
|---|-------------|--------|-----|-----------|
| 0 | 101         | Arun   | 25  | Chennai   |
| 1 | 102         | Bala   | 30  | Mumbai    |
| 2 | 103         | Charan | 28  | Delhi     |
| 3 | 104         | Divya  | 35  | Chennai   |
| 4 | 106         | Eshan  | 40  | Bangalore |

### Step 2: Check and Remove Duplicate Customer IDs

```

print("Duplicate Customer IDs in Transactions:")
print(transactions[transactions.duplicated(subset='Customer_ID', keep=False)])
transactions_clean = transactions.drop_duplicates(
    subset='Customer_ID',
    keep='first'
)

print("After Removing Duplicates:")
display(transactions_clean)
print("Duplicate IDs in Demographics:")
print(demographics[demographics.duplicated(
    subset='Customer_ID',
    keep=False
)])

```

Duplicate Customer IDs in Transactions:

|   | Customer_ID | Transaction_Amount | Transaction_Date |
|---|-------------|--------------------|------------------|
| 4 | 105         | 900                | 2026-08-05       |
| 5 | 105         | 900                | 2026-08-05       |

After Removing Duplicates:

|   | Customer_ID | Transaction_Amount | Transaction_Date |
|---|-------------|--------------------|------------------|
| 0 | 101         | 500                | 2026-08-01       |
| 1 | 102         | 1200               | 2026-08-02       |
| 2 | 103         | 750                | 2026-08-03       |
| 3 | 104         | 300                | 2026-08-04       |
| 4 | 105         | 900                | 2026-08-05       |

Duplicate IDs in Demographics:  
Empty DataFrame  
Columns: [Customer\_ID, Name, Age, City]  
Index: []

### Step 3: Validate Data Using Grouping

```

customer_count = (
    transactions
    .groupby('Customer_ID')
    .size()
    .reset_index(name='Count')
)

print("Customer ID Occurrence Count:")
display(customer_count)
duplicate_customers = customer_count[
    customer_count['Count'] > 1
]

print("Customers with Duplicate Records:")
display(duplicate_customers)

```

Customer ID Occurrence Count:

|   | Customer_ID | Count |
|---|-------------|-------|
| 0 | 101         | 1     |
| 1 | 102         | 1     |
| 2 | 103         | 1     |
| 3 | 104         | 1     |
| 4 | 105         | 2     |

Customers with Duplicate Records:

|   | Customer_ID | Count |
|---|-------------|-------|
| 4 | 105         | 2     |

### Step 4: Merge the DataFrames

```

merged_df = pd.merge(
    transactions_clean,
    demographics,

```

```

        on='Customer_ID',
        how='outer',
        indicator=True
    )

    display(merged_df)

```

|   | Customer_ID | Transaction_Amount | Transaction_Date | Name   | Age  | City      | _merge     |
|---|-------------|--------------------|------------------|--------|------|-----------|------------|
| 0 | 101         | 500.0              | 2026-08-01       | Arun   | 25.0 | Chennai   | both       |
| 1 | 102         | 1200.0             | 2026-08-02       | Bala   | 30.0 | Mumbai    | both       |
| 2 | 103         | 750.0              | 2026-08-03       | Charan | 28.0 | Delhi     | both       |
| 3 | 104         | 300.0              | 2026-08-04       | Divya  | 35.0 | Chennai   | both       |
| 4 | 105         | 900.0              | 2026-08-05       | NaN    | NaN  | NaN       | left_only  |
| 5 | 106         | NaN                | NaN              | Eshan  | 40.0 | Bangalore | right_only |

#### Step 5: Identify Missing or Mismatched Records

```

missing_demographic = merged_df[
    merged_df['_merge'] == 'left_only'
]

missing_transaction = merged_df[
    merged_df['_merge'] == 'right_only'
]

print("Customers Missing Demographic Information:")
display(missing_demographic)

print("Customers Missing Transaction Information:")
display(missing_transaction)

```

Customers Missing Demographic Information:

|   | Customer_ID | Transaction_Amount | Transaction_Date | Name | Age | City | _merge    |
|---|-------------|--------------------|------------------|------|-----|------|-----------|
| 4 | 105         | 900.0              | 2026-08-05       | NaN  | NaN | NaN  | left_only |

Customers Missing Transaction Information:

|   | Customer_ID | Transaction_Amount | Transaction_Date | Name  | Age  | City      | _merge     |
|---|-------------|--------------------|------------------|-------|------|-----------|------------|
| 5 | 106         | NaN                | NaN              | Eshan | 40.0 | Bangalore | right_only |

#### Step 6: Filter Valid Records

```

valid_records = merged_df[
    merged_df['_merge'] == 'both'
]

display(valid_records)
valid_records = valid_records.drop(columns=['_merge'])

```

|   | Customer_ID | Transaction_Amount | Transaction_Date | Name   | Age  | City    | _merge |
|---|-------------|--------------------|------------------|--------|------|---------|--------|
| 0 | 101         | 500.0              | 2026-08-01       | Arun   | 25.0 | Chennai | both   |
| 1 | 102         | 1200.0             | 2026-08-02       | Bala   | 30.0 | Mumbai  | both   |
| 2 | 103         | 750.0              | 2026-08-03       | Charan | 28.0 | Delhi   | both   |
| 3 | 104         | 300.0              | 2026-08-04       | Divya  | 35.0 | Chennai | both   |

#### Step 7: Sort the Data

```

valid_records = valid_records.sort_values(
    by='Customer_ID',
    ascending=True
)

display(valid_records)

```

|   | Customer_ID | Transaction_Amount | Transaction_Date | Name   | Age  | City    |
|---|-------------|--------------------|------------------|--------|------|---------|
| 0 | 101         | 500.0              | 2026-08-01       | Arun   | 25.0 | Chennai |
| 1 | 102         | 1200.0             | 2026-08-02       | Bala   | 30.0 | Mumbai  |
| 2 | 103         | 750.0              | 2026-08-03       | Charan | 28.0 | Delhi   |
| 3 | 104         | 300.0              | 2026-08-04       | Divya  | 35.0 | Chennai |

```
sorted_amount = valid_records.sort_values(
    by='Transaction_Amount',
    ascending=False
)

display(sorted_amount)
```

|   | Customer_ID | Transaction_Amount | Transaction_Date | Name   | Age  | City    |
|---|-------------|--------------------|------------------|--------|------|---------|
| 1 | 102         | 1200.0             | 2026-08-02       | Bala   | 30.0 | Mumbai  |
| 2 | 103         | 750.0              | 2026-08-03       | Charan | 28.0 | Delhi   |
| 0 | 101         | 500.0              | 2026-08-01       | Arun   | 25.0 | Chennai |
| 3 | 104         | 300.0              | 2026-08-04       | Divya  | 35.0 | Chennai |

#### Step 8: Reset Index

```
final_df = valid_records.reset_index(drop=True)

print("Final Combined Data:")
display(final_df)
```

Final Combined Data:

|   | Customer_ID | Transaction_Amount | Transaction_Date | Name   | Age  | City    |
|---|-------------|--------------------|------------------|--------|------|---------|
| 0 | 101         | 500.0              | 2026-08-01       | Arun   | 25.0 | Chennai |
| 1 | 102         | 1200.0             | 2026-08-02       | Bala   | 30.0 | Mumbai  |
| 2 | 103         | 750.0              | 2026-08-03       | Charan | 28.0 | Delhi   |
| 3 | 104         | 300.0              | 2026-08-04       | Divya  | 35.0 | Chennai |

```
final_indexed_df = final_df.set_index('Customer_ID')

display(final_indexed_df)
```

|             | Transaction_Amount | Transaction_Date | Name   | Age  | City    |
|-------------|--------------------|------------------|--------|------|---------|
| Customer_ID |                    |                  |        |      |         |
| 101         | 500.0              | 2026-08-01       | Arun   | 25.0 | Chennai |
| 102         | 1200.0             | 2026-08-02       | Bala   | 30.0 | Mumbai  |
| 103         | 750.0              | 2026-08-03       | Charan | 28.0 | Delhi   |
| 104         | 300.0              | 2026-08-04       | Divya  | 35.0 | Chennai |

#### Step 9: Final Validation

```
print("Duplicate Customer IDs:",
      final_df['Customer_ID'].duplicated().sum())
print("\nMissing Values:")
print(final_df.isnull().sum())
print("\nData Information:")
final_df.info()
```

Duplicate Customer IDs: 0

Missing Values:  
Customer\_ID 0  
Transaction\_Amount 0  
Transaction\_Date 0

```

Name          0
Age           0
City          0
dtype: int64

Data Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Customer_ID           4 non-null     int64
1   Transaction_Amount     4 non-null     float64
2   Transaction_Date       4 non-null     object
3   Name                  4 non-null     object
4   Age                   4 non-null     float64
5   City                  4 non-null     object
dtypes: float64(2), int64(1), object(3)
memory usage: 324.0+ bytes

```

Explanation Filtering: Duplicate and mismatched customer records are identified using duplicated() and Boolean filtering. Grouping: groupby() checks how many times each customer ID occurs. Merging: pd.merge() combines transaction and demographic data. Outer join: Prevents accidental loss of records. Validation: \_merge identifies missing or unmatched entries. Sorting: sort\_values() arranges records systematically. Indexing: reset\_index() creates a clean, continuous index. Debugging: isnull(), duplicated(), and \_merge help locate data problems.

## 2. Complete Data Analysis Pipeline Using Pandas Objective

This pipeline performs:

- Loading multiple files
- Handling corrupted or inconsistent files
- Combining datasets
- Cleaning and processing data
- Filtering based on user conditions
- Grouping and aggregation
- Sorting
- Visualization
- Debugging and validation

### Step 1: Import Required Libraries

```

import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path

```

### Step 2: Upload Multiple Files in Google Colab

```
from google.colab import files
```

```
uploaded = files.upload()
```

No file chosen      Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving sales\_data\_1.csv to sales\_data\_1.csv  
 Saving sales\_data\_2.csv to sales\_data\_2.csv

### Step 3: Efficiently Read Multiple Files

```

dataframes = []

for filename in uploaded.keys():
    try:

```

```

df = pd.read_csv(filename)
dataframes.append(df)
print(f"{filename} loaded successfully")

except UnicodeDecodeError:
    print(f"Encoding problem in {filename}")

except pd.errors.ParserError:
    print(f"Corrupted or improperly formatted file: {filename}")

except Exception as e:
    print(f"Error reading {filename}: {e}")

```

```

sales_data_1.csv loaded successfully
sales_data_2.csv loaded successfully
sales_data_3.csv loaded successfully

```

```

combined_df = pd.concat(
    dataframes,
    ignore_index=True
)

print("Combined Data:")
display(combined_df.head())

```

Combined Data:

|   | Customer_ID | Category    | Amount  | City      | Transaction_Date | customer id | category | amount | city | transaction date |
|---|-------------|-------------|---------|-----------|------------------|-------------|----------|--------|------|------------------|
| 0 | 101.0       | Electronics | 25000.0 | Chennai   | 2026-08-01       | NaN         | NaN      | NaN    | NaN  | NaN              |
| 1 | 102.0       | Clothing    | 3500.0  | Mumbai    | 2026-08-01       | NaN         | NaN      | NaN    | NaN  | NaN              |
| 2 | 103.0       | Groceries   | 1800.0  | Delhi     | 2026-08-02       | NaN         | NaN      | NaN    | NaN  | NaN              |
| 3 | 104.0       | Electronics | 42000.0 | Bangalore | 2026-08-02       | NaN         | NaN      | NaN    | NaN  | NaN              |
| 4 | 105.0       | NaN         | 6500.0  | Chennai   | 2026-08-03       | NaN         | NaN      | NaN    | NaN  | NaN              |

## Step 5: Inspect the Data

```

print("First 5 rows:")
display(combined_df.head())

print("\nLast 5 rows:")
display(combined_df.tail())

print("\nDataset Information:")
combined_df.info()

print("\nStatistical Summary:")
display(combined_df.describe())

```

First 5 rows:

|   | Customer_ID | Category    | Amount  | City      | Transaction_Date | customer id | category | amount | city | transaction date |
|---|-------------|-------------|---------|-----------|------------------|-------------|----------|--------|------|------------------|
| 0 | 101.0       | Electronics | 25000.0 | Chennai   | 2026-08-01       | NaN         | NaN      | NaN    | NaN  | NaN              |
| 1 | 102.0       | Clothing    | 3500.0  | Mumbai    | 2026-08-01       | NaN         | NaN      | NaN    | NaN  | NaN              |
| 2 | 103.0       | Groceries   | 1800.0  | Delhi     | 2026-08-02       | NaN         | NaN      | NaN    | NaN  | NaN              |
| 3 | 104.0       | Electronics | 42000.0 | Bangalore | 2026-08-02       | NaN         | NaN      | NaN    | NaN  | NaN              |
| 4 | 105.0       | NaN         | 6500.0  | Chennai   | 2026-08-03       | NaN         | NaN      | NaN    | NaN  | NaN              |

Last 5 rows:

|    | Customer_ID | Category    | Amount  | City      | Transaction_Date | customer id | category | amount | city | transaction date |
|----|-------------|-------------|---------|-----------|------------------|-------------|----------|--------|------|------------------|
| 25 | 126.0       | Clothing    | NaN     | Mumbai    | 2026-08-13       | NaN         | NaN      | NaN    | NaN  | NaN              |
| 26 | 127.0       | Groceries   | 1600.0  | Delhi     | 2026-08-14       | NaN         | NaN      | NaN    | NaN  | NaN              |
| 27 | 128.0       | Furniture   | 24000.0 | Bangalore | 2026-08-14       | NaN         | NaN      | NaN    | NaN  | NaN              |
| 28 | 129.0       | Electronics | 41000.0 | Chennai   | 2026-08-15       | NaN         | NaN      | NaN    | NaN  | NaN              |
| 29 | 130.0       | Clothing    | 4100.0  | Mumbai    | 2026-08-15       | NaN         | NaN      | NaN    | NaN  | NaN              |

Dataset Information:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 30 entries, 0 to 29

Data columns (total 10 columns):

| # | Column           | Non-Null Count | Dtype   |
|---|------------------|----------------|---------|
| 0 | Customer_ID      | 20 non-null    | float64 |
| 1 | Category         | 19 non-null    | object  |
| 2 | Amount           | 19 non-null    | float64 |
| 3 | City             | 20 non-null    | object  |
| 4 | Transaction_Date | 20 non-null    | object  |
| 5 | customer id      | 10 non-null    | float64 |
| 6 | category         | 10 non-null    | object  |
| 7 | amount           | 10 non-null    | object  |
| 8 | city             | 10 non-null    | object  |
| 9 | transaction date | 10 non-null    | object  |

dtypes: float64(3), object(7)

memory usage: 2.5+ KB

Statistical Summary:

|       | Customer_ID | Amount       | customer id |
|-------|-------------|--------------|-------------|
| count | 20.000000   | 19.000000    | 10.00000    |
| mean  | 115.500000  | 16563.157895 | 115.50000   |
| std   | 10.674613   | 15433.196923 | 3.02765     |
| min   | 101.000000  | 1600.000000  | 111.00000   |
| 25%   | 105.750000  | 3150.000000  | 113.25000   |
| 50%   | 115.500000  | 12000.000000 | 115.50000   |
| 75%   | 125.250000  | 27500.000000 | 117.75000   |
| max   | 130.000000  | 45000.000000 | 120.00000   |

## Step 6: Handle Missing Values

```
print("Missing Values:")
print(combined_df.isnull().sum())
combined_df = combined_df.dropna(
    subset=['Customer_ID', 'Amount']
)
combined_df['Category'] = combined_df['Category'].fillna(
    'Unknown'
)
```

Missing Values:

```
Customer_ID      10
Category         11
Amount           11
City             10
Transaction_Date  10
```

```
customer id      20
category         20
amount           20
city             20
transaction date  20
dtype: int64
```

#### Step 7: Remove Duplicate Data

```
duplicate_count = combined_df.duplicated().sum()

print("Number of Duplicate Rows:", duplicate_count)

combined_df = combined_df.drop_duplicates()
combined_df = combined_df.drop_duplicates(
    subset='Customer_ID',
    keep='first'
)
```

```
Number of Duplicate Rows: 0
```

#### Step 8: Convert Data Types

```
combined_df['Customer_ID'] = pd.to_numeric(
    combined_df['Customer_ID'],
    errors='coerce'
)

combined_df['Amount'] = pd.to_numeric(
    combined_df['Amount'],
    errors='coerce'
)
combined_df = combined_df.dropna(
    subset=['Customer_ID', 'Amount']
)
```

#### Step 9: Apply User-Defined Filtering

```
minimum_amount = float(
    input("Enter minimum transaction amount: ")
)

filtered_df = combined_df[
    combined_df['Amount'] >= minimum_amount
]

print("Filtered Data:")
display(filtered_df)
selected_category = input("Enter category: ")

filtered_df = combined_df[
    (combined_df['Amount'] >= minimum_amount) &
    (combined_df['Category'] == selected_category)
]

display(filtered_df)
```



Enter minimum transaction amount: 15000  
Filtered Data:

|    | Customer_ID | Category    | Amount  | City      | Transaction_Date | customer id | category | amount | city | transaction date |  |
|----|-------------|-------------|---------|-----------|------------------|-------------|----------|--------|------|------------------|--|
| 0  | 101.0       | Electronics | 25000.0 | Chennai   | 2026-08-01       | NaN         | NaN      | NaN    | NaN  | NaN              |  |
| 3  | 104.0       | Electronics | 42000.0 | Bangalore | 2026-08-02       | NaN         | NaN      | NaN    | NaN  | NaN              |  |
| 5  | 106.0       | Furniture   | 15000.0 | Mumbai    | 2026-08-03       | NaN         | NaN      | NaN    | NaN  | NaN              |  |
| 8  | 109.0       | Electronics | 30000.0 | Chennai   | 2026-08-05       | NaN         | NaN      | NaN    | NaN  | NaN              |  |
| 20 | 121.0       | Electronics | 45000.0 | Chennai   | 2026-08-11       | NaN         | NaN      | NaN    | NaN  | NaN              |  |
| 23 | 124.0       | Furniture   | 19000.0 | Bangalore | 2026-08-12       | NaN         | NaN      | NaN    | NaN  | NaN              |  |
| 24 | 125.0       | Electronics | 33000.0 | Chennai   | 2026-08-13       | NaN         | NaN      | NaN    | NaN  | NaN              |  |
| 27 | 128.0       | Furniture   | 24000.0 | Bangalore | 2026-08-14       | NaN         | NaN      | NaN    | NaN  | NaN              |  |
| 28 | 129.0       | Electronics | 41000.0 | Chennai   | 2026-08-15       | NaN         | NaN      | NaN    | NaN  | NaN              |  |

Enter category: Electronics

|    | Customer_ID | Category    | Amount  | City      | Transaction_Date | customer id | category | amount | city | transaction date |  |
|----|-------------|-------------|---------|-----------|------------------|-------------|----------|--------|------|------------------|--|
| 0  | 101.0       | Electronics | 25000.0 | Chennai   | 2026-08-01       | NaN         | NaN      | NaN    | NaN  | NaN              |  |
| 3  | 104.0       | Electronics | 42000.0 | Bangalore | 2026-08-02       | NaN         | NaN      | NaN    | NaN  | NaN              |  |
| 8  | 109.0       | Electronics | 30000.0 | Chennai   | 2026-08-05       | NaN         | NaN      | NaN    | NaN  | NaN              |  |
| 20 | 121.0       | Electronics | 45000.0 | Chennai   | 2026-08-11       | NaN         | NaN      | NaN    | NaN  | NaN              |  |
| 24 | 125.0       | Electronics | 33000.0 | Chennai   | 2026-08-13       | NaN         | NaN      | NaN    | NaN  | NaN              |  |
| 28 | 129.0       | Electronics | 41000.0 | Chennai   | 2026-08-15       | NaN         | NaN      | NaN    | NaN  | NaN              |  |

## Step 10: Group and Aggregate Data

```
summary = (
    filtered_df
    .groupby('Category')
    .agg(
        Total_Amount=('Amount', 'sum'),
        Average_Amount=('Amount', 'mean'),
        Transaction_Count=('Amount', 'count')
    )
    .reset_index()
)

display(summary)
```

|   | Category    | Total_Amount | Average_Amount | Transaction_Count |  |
|---|-------------|--------------|----------------|-------------------|--|
| 0 | Electronics | 216000.0     | 36000.0        | 6                 |  |

## Step 11: Sort the Results

```
summary = summary.sort_values(
    by='Total_Amount',
    ascending=False
).reset_index(drop=True)

display(summary)
```

|   | Category    | Total_Amount | Average_Amount | Transaction_Count |  |
|---|-------------|--------------|----------------|-------------------|--|
| 0 | Electronics | 216000.0     | 36000.0        | 6                 |  |

## Step 12: Generate Visualizations

### A. Line Plot

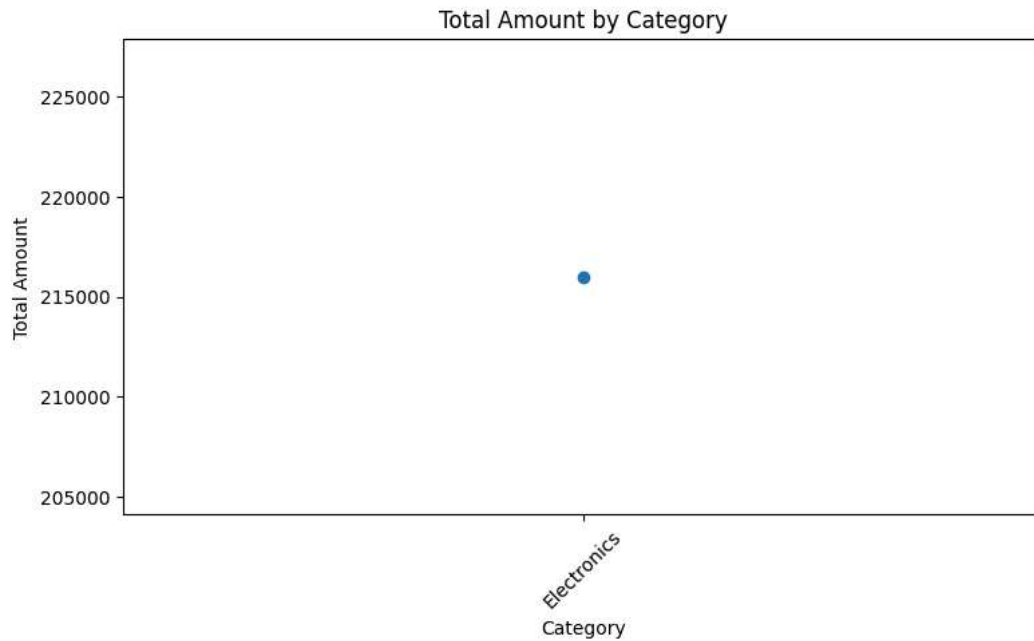
```
plt.figure(figsize=(8, 5))
```

```
plt.plot(
    summary['Category'],
    summary['Total_Amount'],
    marker='o'
)

plt.title('Total Amount by Category')
plt.xlabel('Category')
plt.ylabel('Total Amount')

plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```



## B. Bar Chart

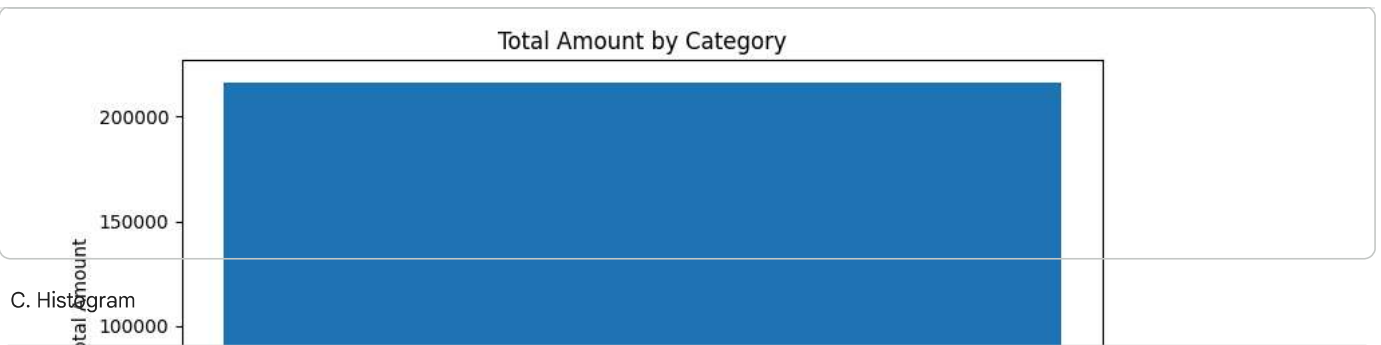
```
plt.figure(figsize=(8, 5))

plt.bar(
    summary['Category'],
    summary['Total_Amount']
)

plt.title('Total Amount by Category')
plt.xlabel('Category')
plt.ylabel('Total Amount')

plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```

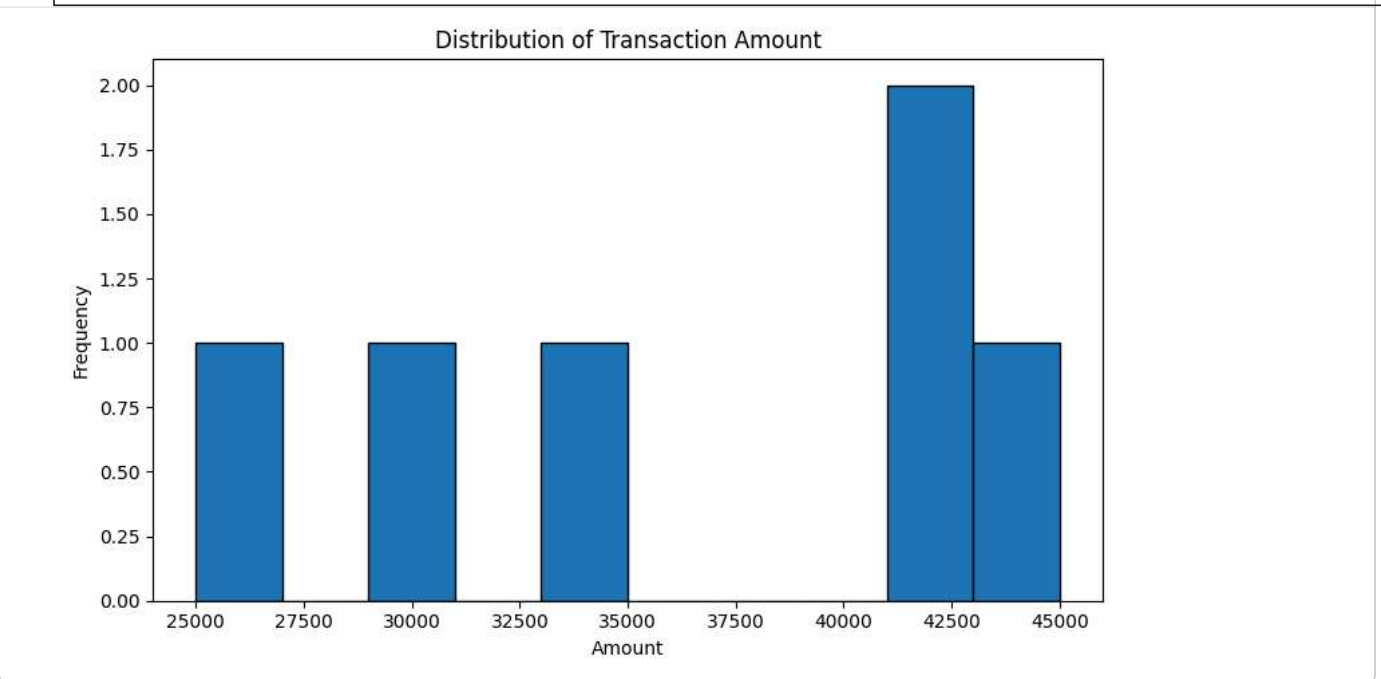


```
plt.figure(figsize=(8, 5))

plt.hist(
    filtered_df['Amount'],
    bins=10,
    edgecolor='black'
)

plt.title('Distribution of Transaction Amount')
plt.xlabel('Amount')
plt.ylabel('Frequency')

plt.tight_layout()
plt.show()
```



Final Validation

```
print("\nFinal Data Information:")
combined_df.info()
print("\nFinal Missing Values:")
print(combined_df.isnull().sum())
print("\nFinal Duplicate Rows:")
print(combined_df.duplicated().sum())
```

```
Final Data Information:
<class 'pandas.core.frame.DataFrame'>
Index: 19 entries, 0 to 29
Data columns (total 10 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   Customer_ID     19 non-null    float64
 1   Category        19 non-null    object
```